Final Project Report: Comparative Study of LSTM, CNN, and SNN Models for EEG-Based Emotion Classification

Althea Luo
yluo344@gatech.edu

Joonhyuk Cha
jcha@gatech.edu

Zuhao Lin
zlin339@gatech.edu

Abstract

Our project studies the effectiveness of LSTM, CNN, and SNN architectures in classifying seven emotions using brain EEG data. Prior work has found CNNs to be successful in detecting emotions using such data. We propose that architectures that have the ability to process series information would be more successful than CNNs for classifying emotions. We tested against an HCNN baseline model and found an LSTM model to be the most successful in classifying emotions.

1. Introduction

Emotion classification is the task of categorizing emotions based on various forms of data produced by humans, such as facial expressions or brain signals. Prior research uses images of faces to classify emotions. However, this may not be able to fully capture the true feelings of an individual, which is not necessarily reflected on an individual's outward appearance. We propose building three deep learning models—a long short-term memory (LSTM) model, a convolutional neural network (CNN), and a spiking neural network (SNN)—that takes in brain electroencephalography (EEG) data as input and returns the corresponding emotion as the classification output. Through this method, we aim to bypass image-dependent data and accurately classify emotions using brain data alone.

Emotion classification is a rapidly expanding area of research with numerous applications throughout industry. For example, it can be a way to gain insights into user perceptions of product designs [3]. In robotics, the recognition and imitation of human emotions is critical in facilitating a more natural human-robot interaction [6] [10] [13]. We expect that our models could be applied to systems in these various fields that require emotion detection and improve their performance.

2. Related Works

Researchers have explored various methods for classifying emotions. Psychologists have long debated how emotions should be defined, with two dominant theories emerging. The discrete emotion theory argues that emotions are distinct categories, typically identifying five to seven basic emotions—such as anger, sadness, and happiness—from which all other emotions derive [2]. In contrast, the dimensional emotion theory views emotions as existing along continuous dimensions, most commonly valence (positive to negative) and arousal (low to high). According to this view, any given emotion can be mapped onto a coordinate system defined by these two axes [9].

Building on this previous psychological research, researchers have collected data and trained models for emotion classification using various techniques. The SEED and DEAP datasets have become foundational in this field, enabling the development of machine learning models such as CNNs, LSTMs, and k-NN, which typically achieve around 70% accuracy [1] [11]. More recently, brain-inspired architectures have led to significant improvements, with some models reaching 80–90% accuracy in emotion identification with Spike-Timing Dependent Plasticity (STDP) [7]. With a hybrid approach using both SNN and CNN, the model was able to reach a peak accuracy of 95% [12].

Similarly to these previous approaches, we also train deep learning models to solve the task. However, our approach differs in that we train three different models and compare their performance. In addition, we only use EEG data as input, whereas other methods typically use images of faces.

3. Method / Approach

We selected three deep learning model architectures to train on the task: a long short-term memory (LSTM) network, a convolutional neural network (CNN), and a spiking neural network (SNN). In the following subsections, we will explain the details of each model, as well as the reasons

for choosing these specific architectures. We also considered other types of models, such as a simple recurrent neural network (RNN) or a transformer model. We ultimately settled on our three models as we thought they were the most appropriate for our task while also spanning a diverse range of architectures.

3.1. LSTM

The first model is a long short-term memory (LSTM) network [3], which is a type of recurrent neural network (RNN). Researchers have applied RNNs for emotion classification with promising results [8]. An LSTM is especially well-suited for tasks requiring the modeling of long-term dependencies in sequential data. Our task and the EEG data that we are working with match these characteristics.

The model consists of multiple LSTM layers, each containing a hidden state, followed by a linear classification layer. We used the Adam optimizer with cross entropy loss, and applied gradient clipping to stabilize the training. Early stopping based on validation loss was used to prevent overfitting.

3.2. CNN

Our next model is a convolutional neural network (CNN) based on the ResNet50 architecture. CNNs have historically been successful when spatial information is crucial. Because we are using EEG data, spatial information can be obtained using the placement of each sensor on the subject's head.

To leverage the strong performance of CNNs in spatial information extraction, the extracted EEG features of 62 channels were mapped onto a 2D array as proposed by Li, Zhang, and He [5]. As there are five frequency bands in our dataset, we used each frequency as its own channel. This resulted in a 3D array of size $5 \times 19 \times 18$, in contrast to the 20×20 2D array used in [5] (note: we are unsure how the authors' method results in their claimed 20×20 feature map). To reach the same input size as the original paper, we used bilinear interpolation to map our 19×18 array to 20×20. We also trained the CNN on each channel individually to compare the performance. We tried both normalized (to mean of zero and variance of one) and un-normalized data and compared the performance between the two.

Additionally, because each trial has a different number of extracted features but only one label per trial, the CNN was trained using the trial containing the extracted feature's label as the feature's label. This must be done because unlike an LSTM, CNNs have no built-in capability to process sequential data. To predict the classification for each input, we used the average outputs of the CNN as the final prediction. We used ReLU as the non-linear activation function.

The CNN architectures will be trained with ADAM and we will experiment with different weight decay values. Cross entropy loss will be used for multi-class classification.

3.3. SNN

For the third model, we implemented a spiking neural network (SNN), which more closely mimics biological neuronal processing, and is particularly well-suited for modeling the temporal and event-driven nature of EEG data. Our SNN utilized a simple two-layer feedforward architecture with Leaky Integrate-and-Fire (LIF) neurons and surrogate gradient backpropagation. The input consisted of the concatenated spike-encoded EEG features. The hidden layer size was varied in our experiments (10, 16, 32, 64, 128), with 16 providing the best stability and performance. The output layer comprised seven LIF neurons, each corresponding to an emotion class.

To experiment dynamic data with the SNN, we employed Poisson spike encoding: each continuous feature was converted into a spike train by generating a binary event at each time step with a probability proportional to its normalized value. This encoding captures both the magnitude and temporal dynamics of the EEG features in a biologically plausible manner. We experimented with various encoding window lengths (T), selecting T=20 as optimal for balancing information content and computational efficiency.

We trained the SNN using the Adam optimizer, a low learning rate (5e-5), and Cross entropy loss for multi-class classification.

4. Data

The dataset that we selected for our experiments is the SEED-VII dataset [4]. It contains brain activity in the form of electroencephalography (EEG) data, recorded from twenty subjects as they watched videos intended to elicit one of seven emotions (happiness, sadness, fear, disgust, surprise, anger, and neutral). This EEG data is provided in raw and preprocessed versions, as well as in the form of differential entropy (DE) features. We chose to use the DE features, and more specifically, the smoothed versions of the features, as we considered them likely to be the most informative and easy to work with.

As the dataset does not provide splits for training, validation, and testing, we created these ourselves with a split ratio of 80%/10%/10%. The DE features are in the shape $(\text{sequence length}, 5, 62)$, where 5 is the number of frequency bands and 62 is the number of EEG channels. The last two dimensions were flattened to change the data into the shape $(\text{sequence length}, 5 \times 62)$ before it was passed into the models.

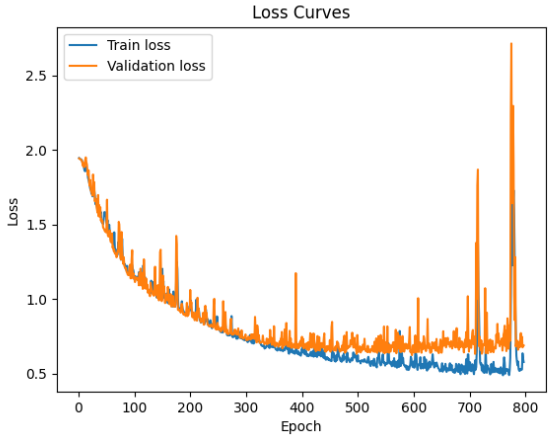


Figure 1. Training and validation losses for the LSTM model.

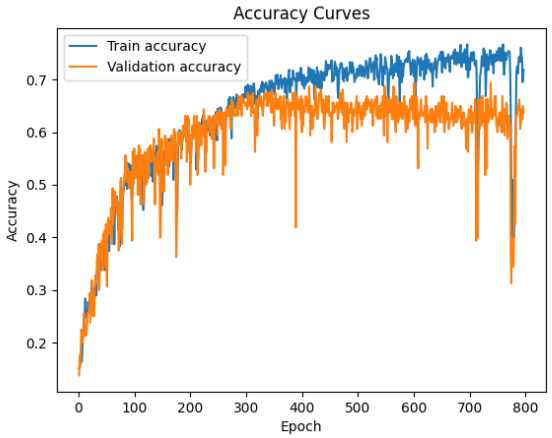


Figure 2. Training and validation accuracies for the LSTM model.

5. Experiments and Results

5.1. LSTM Model

For the LSTM model, we used four LSTM layers with a 256-dimensional hidden state. We trained the model with a batch size of 128 and a learning rate of 1e-4. The gradients were clipped to a max norm of 0.5. These parameters are the result of tuning to balance model performance, training time, and training stability.

Figures 1 and 2 show the loss and accuracy curves for the training process. A trend of improvement and subsequent overfitting can be seen in both graphs. The model corresponding to the lowest validation loss was chosen as the final model and evaluated on the test set, where it achieved 71.2% accuracy. This is a reasonable result, when compared to previous research on this task using LSTMs. It appears that the LSTM was able to accurately capture long-term dependencies in the brain data that were relevant to classifying the emotions.

5.2. CNN Model

We implemented the HCNN [5] as a baseline comparison to our proposed ResNet50-based CNN. As discussed previously, for each trial, we mapped the data to be of shape (sequence length, channels, 20, 20) and all extracted features from the same video have the same label. In total, our dataset shape is (sequence length $\times$ num trials $\times$ num subjects, 5, 20, 20). After initially testing a few hyperparameters, we decided to use the Adam optimizer with a learning rate of 5E-4, weight decay of 5E-3, and ran it for 1E4 Epochs. Adam was used because it has the ability to add momentum and weight decay while also being a widely popular optimizer. We also saved the model with the best validation accuracy over all runs as our final trained model. We normalized the data for each channel and reserved data from subject one as the validation set and data from subject two as the test set.

We found that HCNNs trained using these parameters preformed very poorly, achieving around 20% final test accuracies at the high end. This is only marginally better than randomly guessing classes and is a large discrepancy from the 28%-88% claimed in [5] and the 40%-50% claimed in [4].

The training curves and confusion matrices for these models suggest that most models initially slightly improve from training, but quickly plateau or crater in validation accuracy after a few thousand epochs while continuing to improve in training accuracy. This caused our training routine to select models with limited training as our "best" models as they achieved the highest validation accuracies. This can be seen in the confusion matrices (Figure 10) as these models tend to guess a few classes during inference.

Because the training curves show that the train and validation losses diverged immediately, we believe that the model is immediately overfitting to the training dataset. To address this, we tried various methods to address overfitting. Please refer to section 9.1 for further details.

After hyperparameter tuning, we found an HCNN model trained on un-normalized data to be our best model. This model had a training accuracy of 44.52%, validation accuracy of 26.21%, test accuracy of 27.5%, and a resulting test F1 score of 27.5%. We then trained RESNET50 models using the same un-normalized data. Our RESNET model was initialized using the pretrained RESNET50 IMAGENET1K_V1 weights provided by PyTorch with unfrozen layers. Both the convolution layers and linear layers were trained for our RESNET50 model. Our reasoning is that brain EEG data is very different from image data, which RESNET50 is trained on. As such, the pretrained weights are unlikely to be useful for feature extraction. Our training curves (Figure 4) seem to support this idea as training accuracy starts around 14% (random guessing).

We trained our RESNET50 architecture using the same

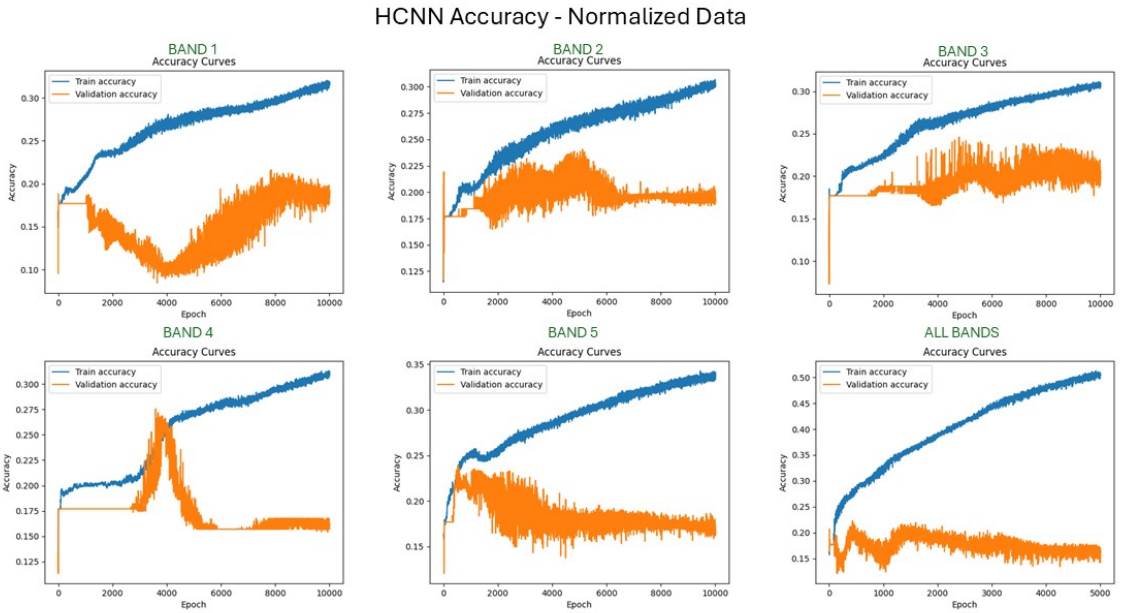


Figure 3. Training and validation accuracies for the HCNN model with normalized data.

parameters except with a max number of epochs of 100. This is due to RESNET50 quickly converging onto the training dataset, with extensive training resulting in little improvement. Un-normalized data was used because it seems to work best for HCNN models. It can be seen in Figure 4 that much like the HCNN models, RESNET50 immediately starts to overfit the training data. This is not a surprising result, as RESNET50 has a very large capacity and was designed to classify 224×224 images of 1000 different classes. As expected, RESNET50 achieved 95+% training accuracy relatively quickly but still did not improve the validation or test accuracy. RESNET50 thus just memorizes the training dataset, which is unhelpful in classifying emotions.

It is possible that the large discrepancy from our HCNN models and the results in [5] was due to the difference in datasets and labels. [5] used another SEED dataset with only three labels: positive, negative, and neutral. This means that an untrained model would have a base accuracy of 33%, while for our SEED-VII dataset, it would have a base accuracy of 14%. Using this baseline accuracy, the HCNN model in [5] had a 0–2.6 times improvement in accuracy over an untrained model. With a 26.25% test accuracy, ours had a 1.875 times improvement over the baseline accuracy.

Our RESNET50 model did not preform better than the baseline HCNN models. As discussed previously, this is likely due to the fact that RESNET50 has too much capacity. Our HCNN model, a relatively small model with only two convolution layers, already overfit our data. Thus, it is to no surprise that RESNET50 would as well. Another possibility for our CNN architectures' poor performance could be the data encoding method. There are only 62 features for each time frame. The encoding method from [5] encodes this onto a 20x20 grid, which has 400 features. This inherently introduces a lot of sparsity in the input data, which may impact CNN performance. Additionally, emotion is temporal and the inability for CNNs to handle series data means it is unable to find temporal relationships within data.

5.3. SNN Model

We conducted a series of experiments to evaluate the performance of a spiking neural network (SNN) on the SEED EEG emotion recognition task. Specifically, we systematically tested different hidden layer sizes (10, 16, 32, 64, 128) and tuned critical neuron hyperparameters including the membrane potential decay factor (beta), the surrogate gradient slope (z), and the spike firing threshold (th). Through extensive experimentation, the best test accuracy we achieved was 63.75% as shown in Figures 5 and 6, using a configuration with 16 hidden units, trained over 600 epochs, with hyperparameters beta=0.65, z=2.2, and threshold=0.8.

Compared with the LSTM, which relied on a much larger architecture consisting of 256 hidden units across four lay-

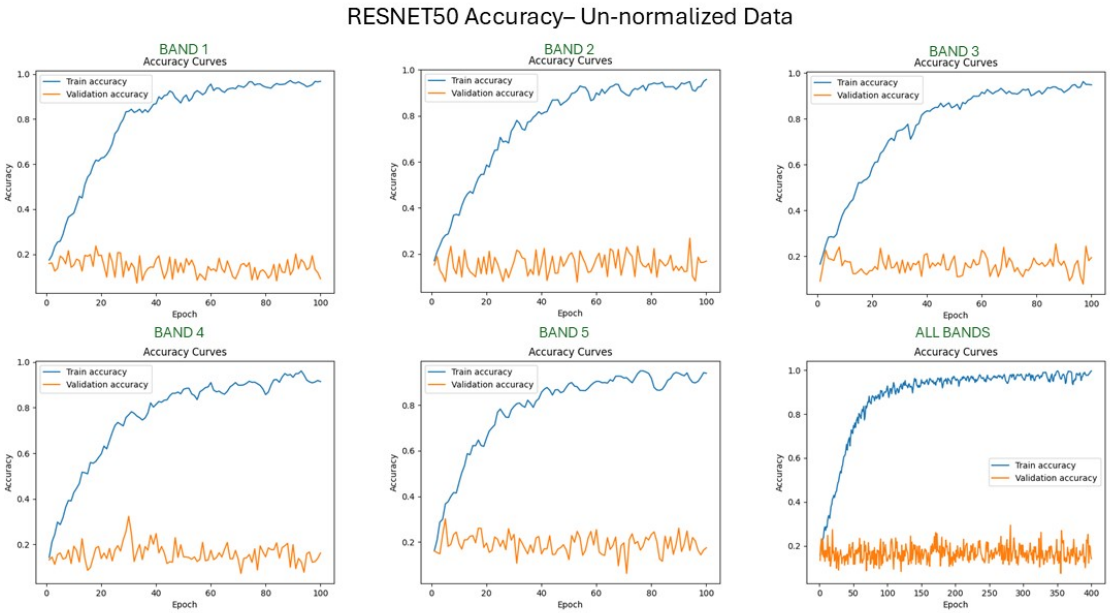


Figure 4. Training and validation accuracies for the RESNET50 models with un-normalized data. Note that the RESNET trained on all bands was trained for four times longer as it had a fourth of the batch size as the other models.

ers, our SNN achieved competitive performance with a much smaller architecture of only 16 hidden units and two layers, highlighting the SNN's efficiency in terms of model size and parameter count.

In our experiments, we also observed that increasing the hidden layer size beyond 16 units consistently led to training instability. Models with larger hidden sizes initially achieved approximately 20% training and validation accuracy within the first 10 epochs. However, after this initial phase, performance sharply deteriorated, collapsing to near-random accuracy around 10–14%, and failing to recover for the remainder of training.

This instability shown in Figures 7 and 8 is likely due to several interacting factors. First, spiking neural networks are particularly sensitive to vanishing and exploding gradients. As the number of neurons increases, the cumulative effect of backpropagating surrogate gradients—approximations of the true, non-differentiable spike function—becomes amplified. Larger networks are more prone to unstable gradient dynamics, which can either cause weight updates to vanish or explode. Second, larger hidden layers can cause issues in spike propagation. In wider networks, neurons may either saturate with excessive firing, resulting in noisy, non-informative learning signals, or fail to fire adequately, leading to a large fraction of "dead" neurons. Both scenarios degrade the ability of the network to form meaningful internal representations over training. Third, conventional deep learning models often use explicit regularization and normalization strategies to stabilize deeper or wider networks. In contrast, our vanilla feedforward SNN lacked such mechanisms. Without stabilization, wide spiking networks become increasingly sensitive to small changes in input, parameter initialization, and gradient flow, leading to rapid training collapse after initial learning. Finally, the limited temporal resolution may have

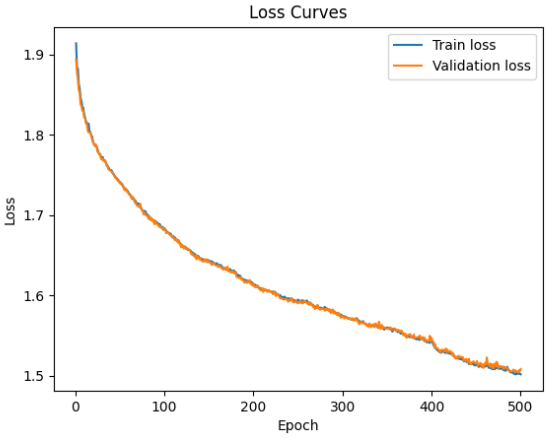


Figure 5. Training and validation losses for the SNN model.

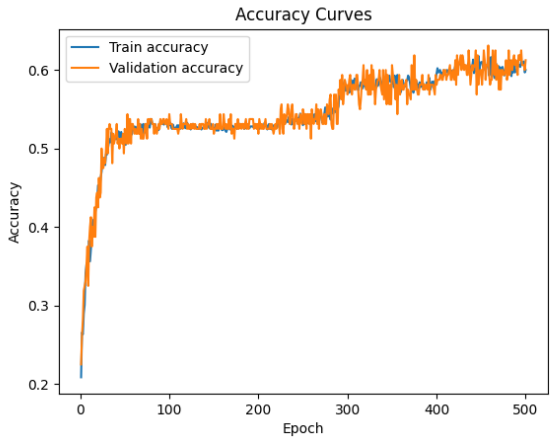


Figure 6. Training and validation accuracies for the SNN model.

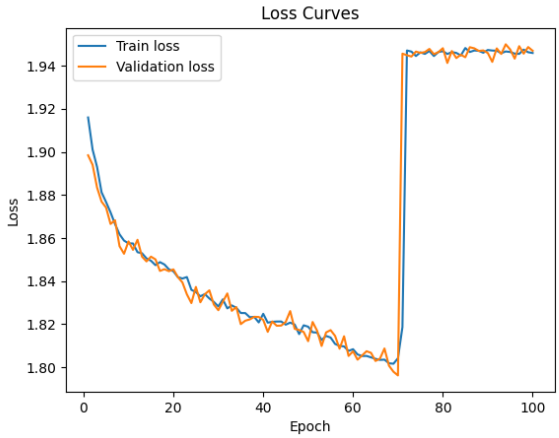


Figure 7. Unstable Training and validation losses for the SNN model.

contributed to the instability. With a Poisson encoding window of only 20 time steps, the network may not have had sufficient opportunity to propagate meaningful temporal dynamics through the increased number of neurons. Wider networks may require either longer simulation windows or alternative encoding schemes to fully leverage their capacity.

6. Conclusion

In conclusion, our LSTM model was the most successful in predicting emotion classifications over EEG data collected for an entire video. Our SNN preformed the second best and our CNN models performed the worst by far. We suspect that the inability of CNNs to process temporal data effectively was the main reason for their ineffectiveness. This is unsurprising, as CNNs are designed for spatial, structured data while EEG data is more temporal.

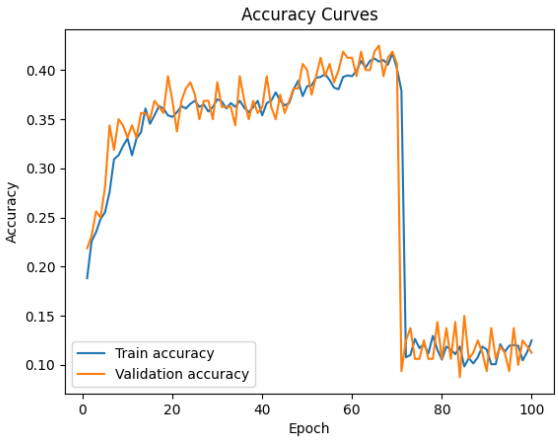


Figure 8. Unstable Training and validation accuracies for the SNN model.

Despite achieving slightly lower accuracy compared to the LSTM model, the SNN demonstrated competitive performance with a much smaller architecture. Our best SNN configuration reached approximately 64% test accuracy, compared to 71% for the LSTM, while using only 2 layers and 16 hidden units. In contrast, the LSTM required a significantly larger network with 4 layers and 256 hidden units. This highlights the SNN's ability to achieve reasonably strong results with a far smaller model size, suggesting potential advantages in scenarios where computational efficiency, memory footprint, or energy consumption are critical factors.

Several recent studies have reported high accuracy for EEG-based emotion recognition using spiking neural networks and hybrid models. For example, the Fractal-SNN approach achieved over 80–90% accuracy on datasets such as DEAP, DREAMER, and SEED-IV, while SNN-EEG frameworks using NeuCube architectures also reported accuracies above 90% [7]. However, it is important to note that these results were primarily obtained on three-class emotion classification tasks and often under subject-dependent protocols, which are substantially easier than the setting addressed in our project. In contrast, our work focused on the more challenging seven-class classification task using the SEED dataset, with an emphasis on multiclass prediction across a broader emotional spectrum. Achieving approximately 60 to 70% accuracy on this harder problem demonstrates that our models perform competitively relative to the complexity of the task.

Possible future work is to effectively integrate the outputs of the three models—LSTM, CNN, and SNN—for emotion classification; they currently operate independently without a unified framework to combine their strengths. Existing hybrid modeling approaches that combine multiple architectures could be explored.

7. Discussion on Deep Learning Aspects

Our problem, classifying emotions given EEG data, involves temporal data over a number of different subjects. The dataset was pre-processed by the authors, who extracted differential entropy (DE) features with five different frequencies. Our models then encoded this data in different ways to suit their needs. The LSTM directly used the entire series of data (with padding) as its input for each video, the CNN mapped the features to a 2D plane, and the SNN used Poisson spike encoding.

Because we knew that the data was temporal, we believed LSTMs to be a good choice as it is known to perform well on temporal tasks such as speech recognition. CNNs were used because our dataset uses EEG data, where each feature is measured at a fixed location on the subject's head. We thought that mapping these features back to a similar position in 2D would allow CNNs, which are best at finding positional relationships, to extract good spatial features. Finally, because we know emotions come from the brain (hence why we are measuring brain data), we chose SNNs because they work similarly to neurons in the brain (with neurons accumulating potential before "firing"). Our thought was that because we are working with brain data, it would make sense to use a deep learning model that more closely mimics the brain.

For the LSTM, the hidden layers and the linear classification layer had parameters to be optimized using an optimizer. For CNNs, both convolution and linear layers had to be optimized. The SNN also has layers to be optimized.

We decided to use the PyTorch framework because it is the framework that is the most familiar for us (every ML class that we know of uses PyTorch at GT). We decided on the Adam optimizer because it is the most popular optimizer, incorporates momentum and weight decay, and has been proven to work for many other models. Cross entropy loss was used as our criterion as we solving a classification problem.

[5] provided us with a CNN baseline to use. Although we did not have the code, the paper has a figure with details on how to build their HCNN model. [5], however, had unreproducible encoding as discussed previously. The authors claim to result in a 20x20 map, but using their method, we could only achieve a 19x18 map. Furthermore, when using the described HCNN, we did not come close to achieving the accuracy reported. This could be due to a variety of reasons. It could simply be that the authors trained many more models than us and found a well working one. It could also be that their dataset worked with HCNN but ours did not.

8. Team Contributions

The contributions of each team member are summarized in Table 1.

References

[1] J Chen, Tony Ro, and Zhigang Zhu. Emotion recognition with audio, video, eeg, and emg: A dataset and baseline approaches. *IEEE Access*, 10:13229–13242, 01 2022. 1

[2] Paul Ekman. An argument for basic emotions. *Cognition and Emotion*, 6:169–200, 05 1992. 1

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, Nov. 1997. 1, 2

[4] Wei-Bang Jiang, Xuan-Hao Liu, Wei-Long Zheng, and Bao-Liang Lu. Seed-vii: A multimodal dataset of six basic emotions with continuous labels for emotion recognition. *IEEE Transactions on Affective Computing*, 2024. 2, 3

[5] Jinpeng Li, Zhaoxiang Zhang, and Huiguang He. Hierarchical convolutional neural networks for eeg-based emotion recognition. *Cognitive Computation*, 10, 04 2018. 2, 3, 4, 7, 8

[6] Zhentao Liu, Min Wu, Weihua Cao, Luefeng Chen, Jianping Xu, Ri Zhang, Mengtian Zhou, and Junwei Mao. A facial expression emotion recognition based human-robot interaction system. *IEEE CAA J. Autom. Sinica*, 4(4):668–676, 2017. 1

[7] Yuling Luo, Qiang Fu, Juntao Xie, Yunbai Qin, Guopei Wu, Junxiu Liu, Frank Jiang, Yi Cao, and Xuemei Ding. Eeg-based emotion classification using spiking neural networks. 8:46007–46016, 03 2020. 1, 6

[8] Qamar Nawaz, Mahzaib Younas, Isma Hamid, and Syed. Recurrent neural network based human emotion recognition using eeg brain signals. *IEEE*, pages 117–122, 09 2023. 2

[9] Jonathan Posner, James A. Russell, and Bradley S. Peterson. The circumplex model of affect: An integrative approach to affective neuroscience, cognitive development, and psychopathology. *Development and Psychopathology*, 17, 09 2005. 1

[10] Amirmohammad Radmehr, Milad Asgari, and Mehdi Tale Masouleh. Experimental study on the imitation of the human head-and-eye pose using the 3-dof agile eye parallel robot with ROS and mediapipe framework. *CoRR*, abs/2111.00452, 2021. 1

[11] Xiaohu Wang, Yongmei Ren, Ze Luo, Wei He, Jun Hong, and Yinzhen Huang. Deep learning-based eeg emotion recognition: Current trends and future perspectives. *Frontiers in Psychology*, 14, 02 2023. 1

[12] FeiFan Xu, Deng Pan, Haohao Zheng, Yu Ouyang, Zhe Jia, and Hong Zeng. Eescn: A novel spiking neural network method for eeg-based emotion recognition. *Computer Methods and Programs in Biomedicine*, 243:107927–107927, 01 2024. 1

[13] Li Zhang, Ming Jiang, Dewan Farid, and M.A. Hossain. Intelligent facial emotion recognition and semantic-based topic detection for a humanoid robot. *Expert Systems with Applications*, 40(13):5160–5168, 2013. 1

Student Name	Contributed Aspects	Details
Althea Luo	SNN implementation and experiment analysis	Implemented and experimented with the SNN model and worked on related sections and literature review in the report.
Joonhyuk Cha	LSTM implementation, model training code, experiment analysis	Implemented the LSTM model and training code and worked on related sections in the report.
Zuhao Lin	CNN models implementation and experiment analysis	Implemented HCNN and RESNET50 architecture and wrote section on methods and results for CNN models.

Table 1. Contributions of team members.

9. Appendix

9.1. CNN Hyperparameter tuning

The confusion matrix for HCNNs trained on normalized data can be seen in 10 and the corresponding training and validation loss can be seen in 9. We tried several different hyperparameter tuning methods to increase the accuracy of our CNN model. We used different weight decay terms and trained on the fifth channel of data only, which [5] finds to be most useful for training. We tested the values 5E-4, 5E-3, and 1E-2, but found little improvement in validation loss as seen in 11. Interestingly, a weight decay value of 5E-2 prevented the loss from immediately diverging but also prevented the HCNN model from improving in general.

Since tweaking the weight decay did not produce the desired result, we then tried various learning rates (also on channel five for consistency). This also did not improve our validation accuracy (no figures provided).

We also used un-normalized data as the above methods did not produce a favorable result. Surprisingly, although un-normalized data did not improve the model when trained on each band individually, it did slightly improve the model trained on all five bands. We tried to reproduce this by training a few more models on the same data but specified the seeds used. We produced a similar model using seed 33, which achieved a test accuracy of 26.25% but a final validation accuracy of 15.97%. This seems to suggest that EEG data between subjects vary wildly and training on one person's data does not necessarily generalize well to others. The loss and accuracy curves can be seen in 12 and 13 and the confusion matrix can be seen in 14.

A summary of each model's performance can be seen in 15, 16, and 17.

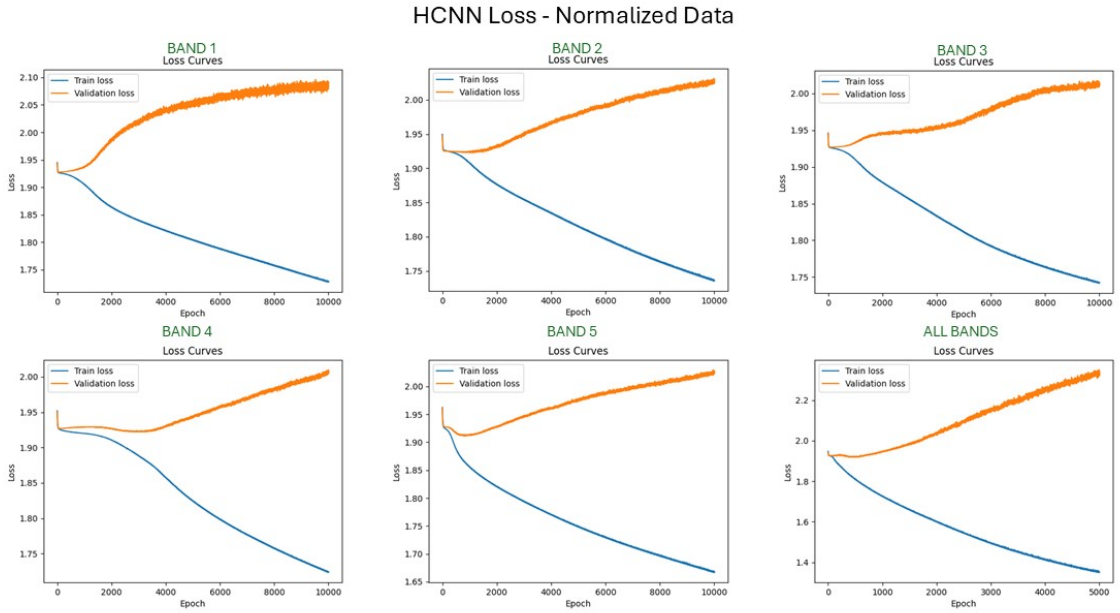


Figure 9. Loss for HCNN with normalized data.

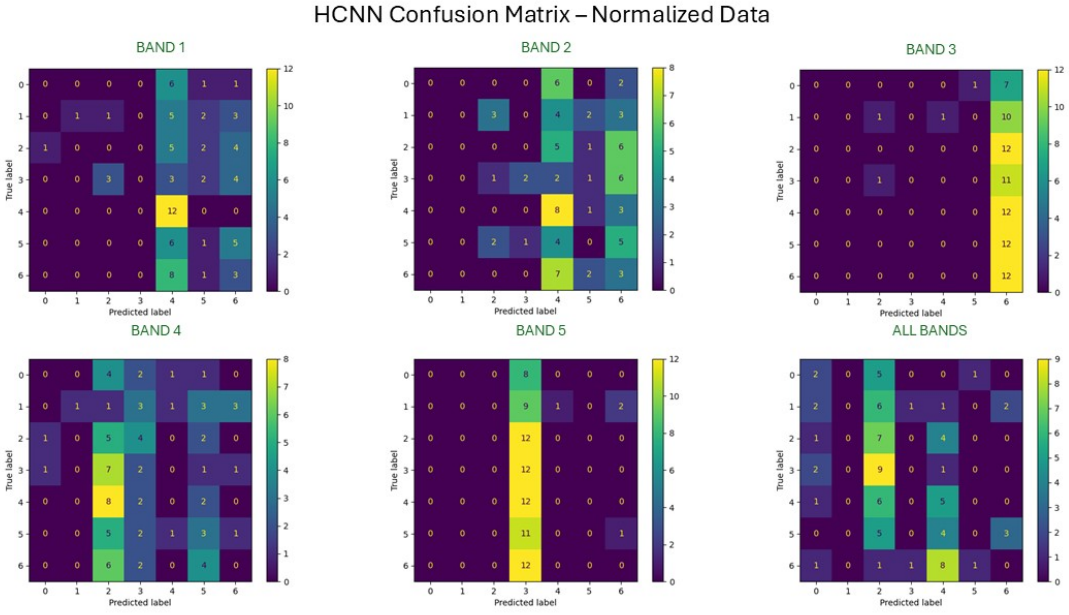


Figure 10. Confusion matrix for HCNN with normalized data.

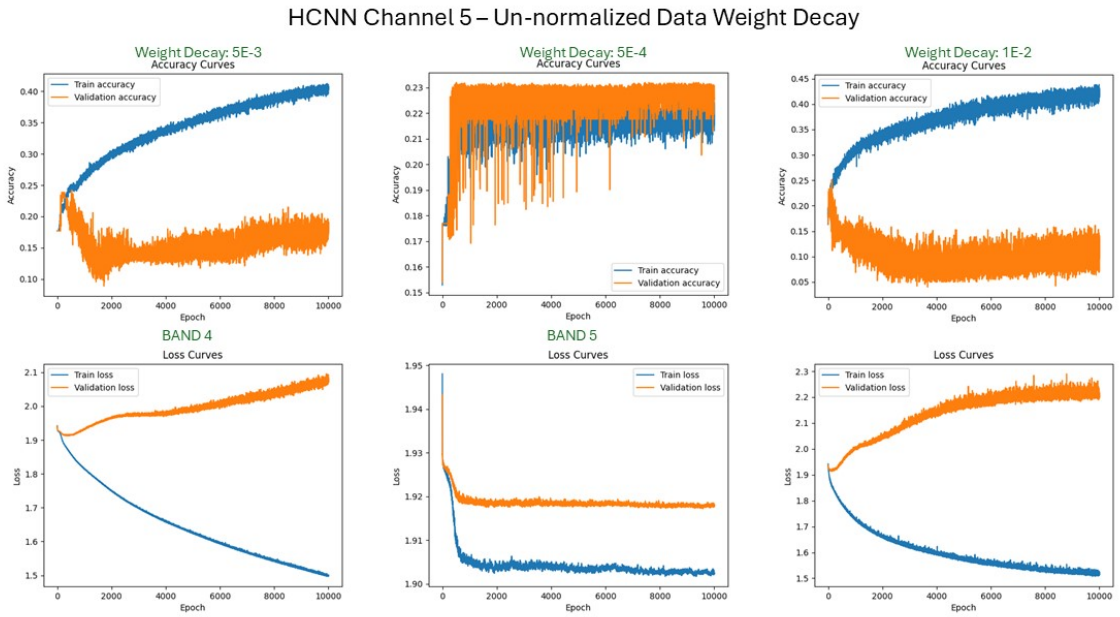


Figure 11. Accuracy and training loss for various weight decay values.

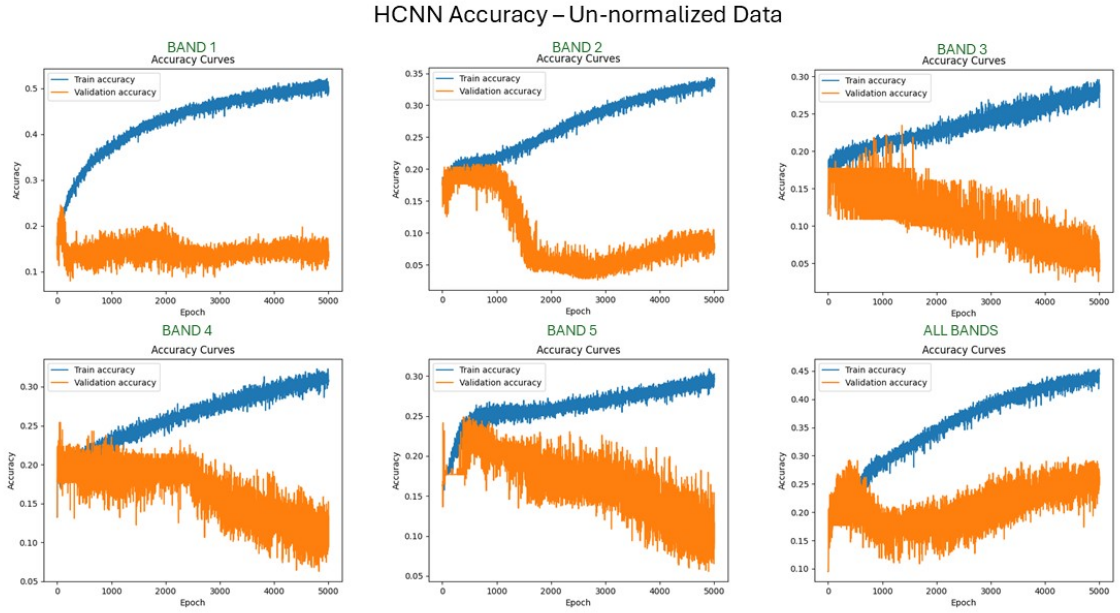


Figure 12. Accuracy for HCNN models trained on un-normalized data.

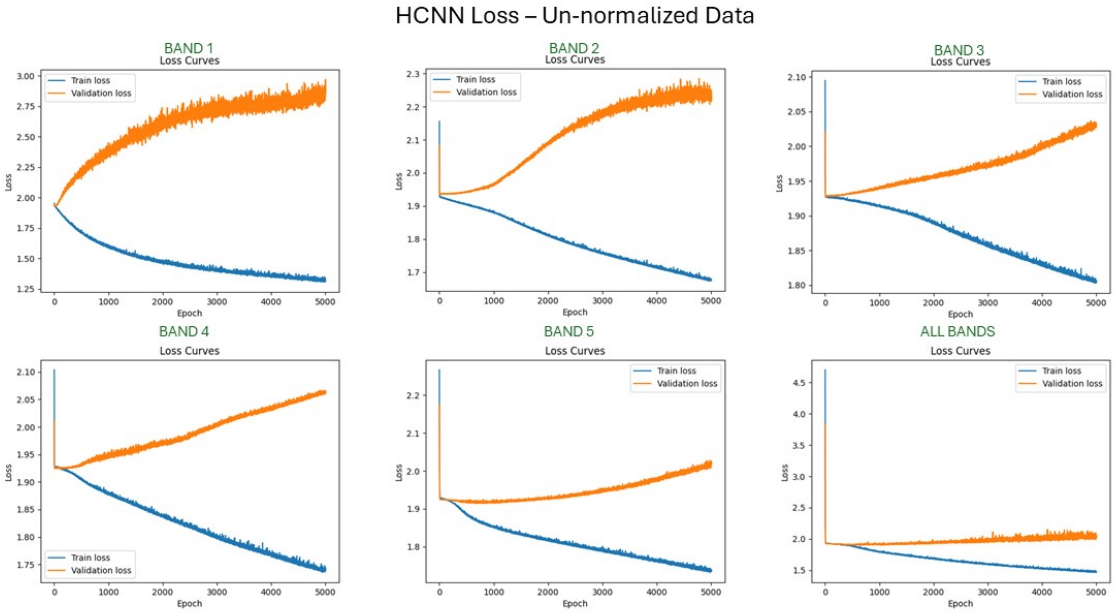


Figure 13. Loss for HCNN models trained on un-normalized data.

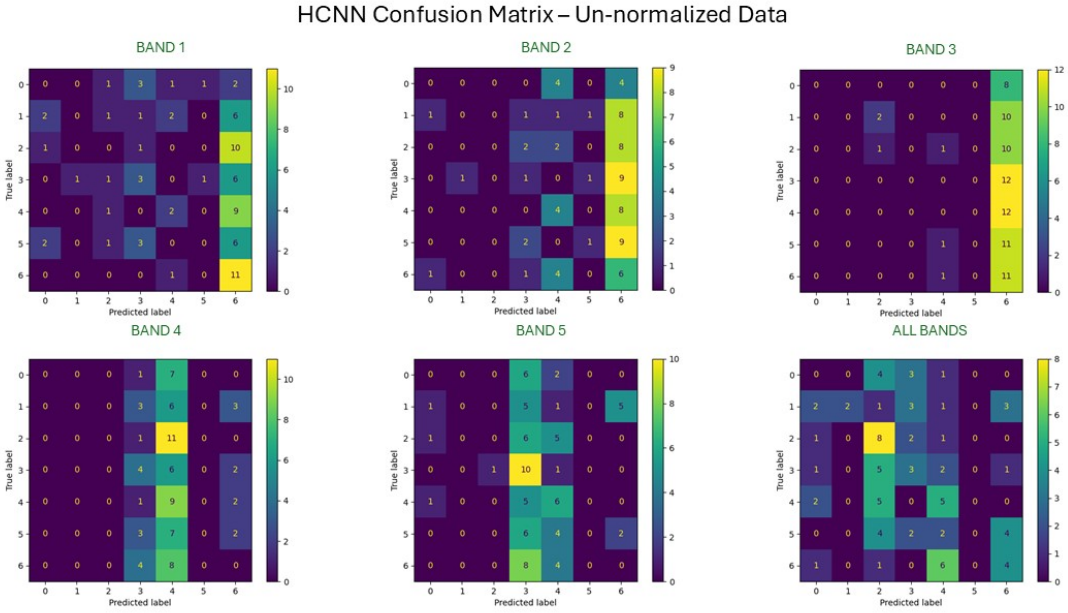


Figure 14. Confusion matrix for un-normalized HCNN models.

Bands	Validation Accuracy	Test Accuracy	Validation Loss	Test Loss	Test F1
BAND 1	13.22%	20.00%	2.8258	2.7530	0.2000
BAND 2	8.20%	15.00%	2.2332	2.1676	0.1500
BAND 3	5.45%	15.00%	2.0312	2.1478	0.1500
BAND 4	13.36%	16.25%	2.0645	2.1198	0.1625
BAND 5	11.64%	20.00%	2.0241	1.9092	0.2000
ALL BANDS	26.21%	27.50%	2.0230	2.0162	0.2750

Figure 15. Performance of HCNN models trained on un-normalized data.

Bands	Validation Accuracy	Test Accuracy	Validation Loss	Test Loss	Test F1
BAND 1	18.90%	21.25%	2.0902	1.9269	0.2125
BAND 2	19.41%	16.25%	2.0307	2.0384	0.1625
BAND 3	19.73%	15.00%	2.0140	2.2589	0.1500
BAND 4	16.17%	13.75%	2.0063	1.9928	0.1375
BAND 5	17.29%	15.00%	2.0240	2.1004	0.1500
ALL BANDS	16.17%	17.50%	2.3470	2.1025	0.1750

Figure 16. Performance of HCNN models trained on normalized data.

Bands	Validation Accuracy	Test Accuracy	Validation Loss	Test Loss	Test F1
BAND 1	14.11%	17.50%	4.2452	3.0154	0.1750
BAND 2	16.89%	12.50%	4.1690	3.9011	0.1250
BAND 3	19.39%	11.25%	3.9949	4.3214	0.1125
BAND 4	16.23%	12.50%	3.7449	3.6908	0.1250
BAND 5	17.49%	11.25%	4.2444	4.0540	0.1125

Figure 17. Performance of RESNET50 models trained on un-normalized data.